COSC 416 April 2026

Okanagan College, Okanagan (BC) Canada

SciKey
Technical Report

Project Sponsor: Gaetan Hains

Université Paris-Est Créteil

John Cortez, Brandon Hay & James Midtdal

Table Of Contents

Project Outline **3**
- Project Description 3
- Data Origins 3
- HAL Access Speed 3

Tuning Parameters **4**

Physical Configuration **6**
- Resource Allocation 6
- I/O Optimization 8

DDL Schema **9**
- Partitioning Strategy & Query Pruning 9
  - Range Partitioning — keyword 9
  - Composite List × Hash Partitioning — document 9
  - Reference Partitioning — doc_keyword, doc_author, author_organism 9
- Advanced Indexing 10
  - B-Tree vs. Bitmap 10
  - Non-Clustered Indexes 10
  - LOCAL Partitioned Indexes 10
- Storage Structures: 11
  - Data Tablespace — scikey_data 11
  - Index Tablespace— scikey_index 11

Data Loading & Stress Testing **12**
- Data Gathering 12
- Bulk Loading 12
- Workload Simulation: 12

Security Roles **16**

Performance Tuning Results **17**
- Partitions & Indexes 17
- Connection Pool 18
- Redo Log Writes 20
- Huge Pages 23

Project Outline

Project Description

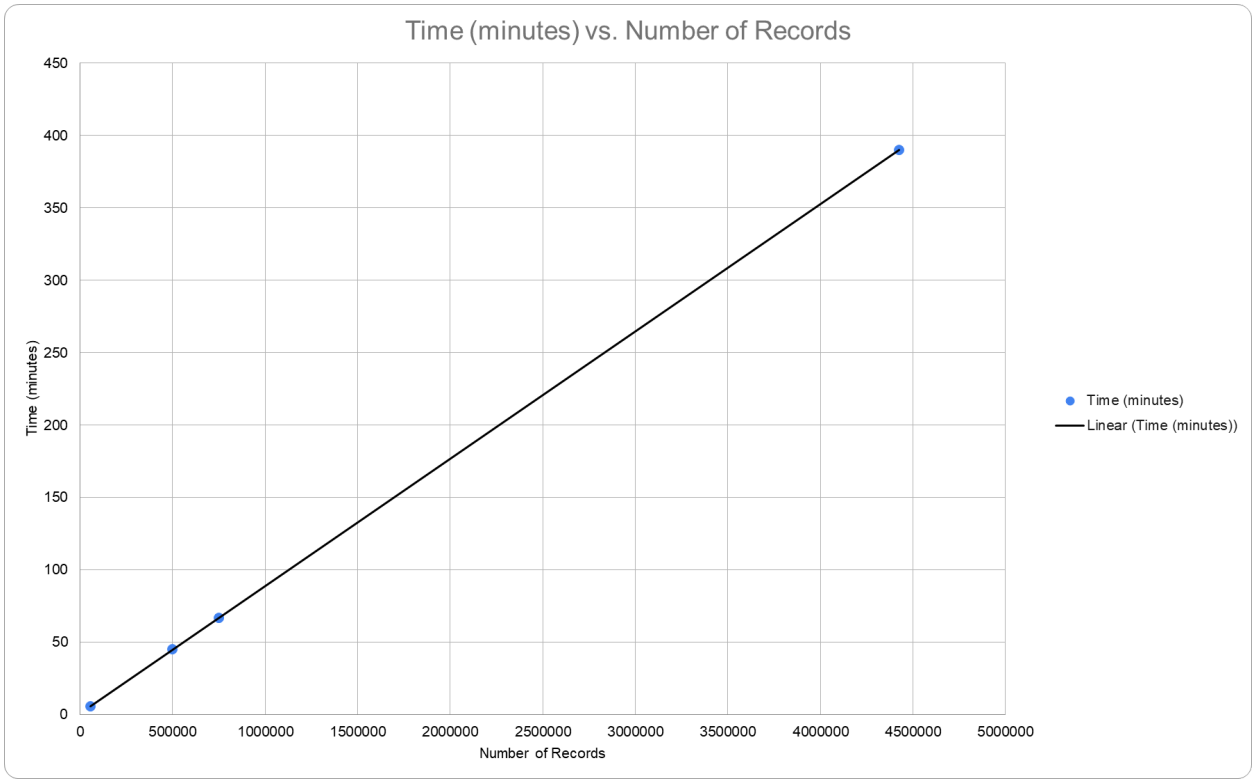
SciKey (Scientific Keywords) is an AI-driven system that automates the extraction, disambiguation, and alignment of keywords from French scientific publications. The project addresses a fundamental challenge in academic research infrastructure: HAL, France's national open-access repository, contains over 3.76 million references and nearly 1.4 million full-text documents, yet the keywords assigned to these publications are author-generated, inconsistent, and often ambiguous. A single term like "CVD" might refer to Chemical Vapor Deposition in a materials science paper or cardiovascular disease in a medical context, and a niche term like "bicrystal" may yield no structured results at all. SciKey resolves this ambiguity by mapping extracted keywords to precise Wikidata concepts, producing a clean, machine-readable knowledge graph of French scientific output. The project is co-supervised by UPEC Library IT staff and Prof. Gaétan Hains, and integrates coursework from Okanagan College's Topics in Databases (COSC 416) into a single end-to-end deliverable.

Data Origins

The data used in this project were collected from HAL (Hyper Articles en Ligne), France's national open-access repository for scholarly publications, managed by the Centre pour la communication scientifique directe (CCSD). HAL aggregates research outputs, including journal articles, conference papers, theses, and preprints from French research institutions and their international collaborators. The repository exposes a public search API built on Apache Solr, which supports structured queries, field-level filtering, faceted search, and cursor-based pagination for programmatic harvesting of metadata at scale.

The original scripts provided by the Lengara College team were slightly modified to retrieve records from both the UPEC portal and the entirety of HAL. Restrictions on keywords and language were removed, ensuring a large, multilingual dataset. Approximately 750,000 records were retrieved from HAL, with an additional 55,712 records from the UPEC portal. Combined, the dataset totals 3.3741 gigabytes in size.

HAL Access Speed



Number of Records	Time (minutes)
55,712	6
500,000	45.1333
750,000	67
4,425,302	390.0466797

(Time for 4,425,302 records extrapolated from actual data)

Tuning Parameters

Parameter	Setting	Impact
sga_target	7,680 MB (~50% of 15.3 GB RAM)	Sets a target for the System Global Area; Oracle automatically sizes the Buffer Cache, Redo Log Buffer, and Shared Pool within this limit
pga_aggregate_target	1,536 MB (~10% of RAM)	Controls memory available for per-session operations like sorting, hashing, and join processing to avoid spilling to disk
Huge Pages	2 MB pages (3,830 pages allocated)	Increases OS page size from 4 KB to 2 MB, reducing TLB misses, preventing Oracle memory from being swapped to disk, and lowering OS page management overhead
Redo Log Size	300 MB → 600 MB (3 groups)	Increases the capacity of each redo log group, reducing the frequency of log switches and the associated checkpoint overhead
DRCP Brokers	2 brokers (optimal)	Controls the number of connection broker processes that manage pooled connections on behalf of clients
DRCP Pool Size	40 connections (default, optimal)	Limits the number of pre-established reusable database sessions, reducing the overhead of repeatedly creating and

		tearing down connections
Partitioning	Range (keyword), Composite List×Hash (document), Reference (bridge tables)	Divides large tables into smaller segments so Oracle can prune irrelevant partitions at query time, scanning only the data that matches the filter
Indexing	B-Tree on high-cardinality columns; Bitmap on low-cardinality columns; all LOCAL on partitioned tables	Provides fast lookup structures. B-Trees for equality/range scans, Bitmaps for bitwise multi-condition filtering. Aligned to partitions for pruning
Tablespaces	scikey_data: 10 GB (autoextend to 25 GB); scikey_index: 2 GB (autoextend to 5 GB)	Physically separates data and index storage into dedicated datafiles so concurrent table scans and index scans don't compete for I/O
Redo/Control File Multiplexing	Two locations (`/u01`, `/u02`)	Maintains redundant copies of redo logs and control files in separate directories for backup and recovery purposes
Undo Tablespace	5 GB	Provides rollback storage for concurrent transactions, ensuring read consistency and supporting transaction isolation

Physical Configuration

Resource Allocation

Approximately 60% of the server's available RAM was allocated to the Oracle instance, with the remaining 40% reserved for the operating system to ensure background processes never risk the instance spilling into swap space. The server has 15.3 GB of total RAM, with sga_target set to 7680M and pga_aggregate_target set to 1536M.

A higher PGA amount was allocated for sorting and hash joins. While our typical queries may not return huge numbers of rows, the database acts as the source of truth for all frontend web applications, meaning it must be able to query and sort large numbers of rows without spilling sorts to disk.

Because we are using Automatic Shared Memory Management, we set a target for the System Global Area and let Oracle size the Database Buffer Cache, Redo Log Buffer, and Shared Pool.

The operating system was reconfigured to use 2 MB Huge Pages. 3850 Huge Pages were set aside in Linux, and Oracle grabbed 3830 of them, resulting in approximately 7660 MB of huge pages allocated — only 40 MB short of our total sga_target. The default page size on Oracle Linux (x86_64) is 4 KB. Using huge pages resulted in 3830 pages dedicated to Oracle versus approximately 1,920,000 pages at the default size. Huge pages were selected to improve Translation Lookaside Buffer (TLB) hit rates, improve reliability since huge pages cannot be swapped to disk, and reduce operating system overhead with page management.

```
[james@cosc416-ora-james ~]$ grep HugePages /proc/meminfo
AnonHugePages:         0 kB
ShmemHugePages:        0 kB
FileHugePages:         0 kB
HugePages_Total:    3850
HugePages_Free:       14
HugePages_Rsvd:        6
HugePages_Surp:        0
```

Figure 1.

I/O Optimization

Redo logs and control files were multiplexed across two locations for redundancy:

- /u01/oradata/
- /u02/oradata/

Unfortunately, all files are on the same physical storage drive due to limited resources. Ideally, they would be on separate drives using Oracle Automatic Storage Management (ASM) for striping and mirroring. Frequently accessed files would be placed on high-speed SSDs, with trace files, logs, and other low-priority files placed on slower SSDs or mechanical drives. Logical separation of redo and control log files still provides a clear structure for backup and recovery, although disk I/O contention remains an issue.

Redo logs were resized from 300 MB to 600 MB to reduce log switching during heavy workloads, with the default number of log groups (3) being maintained.

Undo and temp tablespaces were placed in default locations. The Pluggable Database was allocated a 5 GB undo tablespace to accommodate concurrent transactions and avoid undo contention, while the CDB undo remains minimal since it only handles root-level operations.

SCIKEY_DATA was allocated 10 GB, as our data was approximately 3.07 GB and extra space was needed for materialized views. Extra space also allows for more HAL data to be added to the dataset. SCIKEY_INDEX was allocated 2 GB. Both tablespaces use auto-extend, allowing for growth as data volume increases or if query analysis identifies a need for additional indexes.

DDL Schema

Partitioning Strategy & Query Pruning

The schema employs three partitioning strategies across its tables, each chosen to match the data distribution and access patterns of that table.

Range Partitioning — `keyword`

The `keyword` table is range partitioned on `keyword_text` with one partition per letter A–Z plus a catch-all for numeric, Cyrillic, and non-Latin entries. When a query filters on `keyword_text`, Oracle prunes directly to the relevant letter partition. For example, a search on keywords beginning with 'S' skips 24 of 25 partitions and scans only the 81k-row `p_kw_s` segment. The LOCAL B-Tree indexes `idx_keyword_text` and `idx_keyword_text_upper` then operate exclusively within that segment.

Composite List × Hash Partitioning — `document`

The `document` table uses two-level partitioning. The first level partitions by LIST on `discipline`, separating rows into named discipline partitions with subpartition counts scaled to data volume — Computer Science receives 8 subpartitions for its 133k rows while smaller disciplines like Marketing receive only 2. The second level applies HASH on `hal_document_id` within each discipline partition, evenly distributing rows to prevent skew and improve parallel scan performance. Queries filtering on `discipline` prune to a single LIST partition before any index or subpartition scan occurs, which also confines the bitmap index operations on `document_type` and `discipline` to a single partition segment.

Reference Partitioning — `doc_keyword`, `doc_author`, `author_organism`

The three bridge tables inherit the `document` partition structure through their foreign key chain. This means when a query joins a document to `doc_author` or `doc_keyword` with a discipline filter, Oracle prunes both sides of the join to the same partition simultaneously — the join never crosses partition boundaries. `author_organism` extends this one level further, inheriting through `doc_author` all the way back to `document`.

Advanced Indexing

As this database is designed with general purpose in mind, many different queries may be run on it. The Indexes decided upon have been created to fit the use cases of many possible queries that will be run against the database.

B-Tree vs. Bitmap

B-Tree indexes were applied to all high-cardinality columns such as `hal_id_s`, `doi_id`, and foreign key columns across the junction tables. These allow Oracle to perform efficient equality lookups and range scans in O(log n) time.

Bitmap indexes were applied to low-cardinality columns such as document_type, `discipline`, and `quality` on the documents table. Rather than a tree traversal, Oracle stores one bit vector per distinct value and combines filters using fast bitwise operations. This makes multi-condition filtering on these columns highly efficient. Since this schema serves a read-heavy analytical workload rather than a write-heavy environment, the write-contention limitations of bitmap indexes are not a concern.

Non-Clustered Indexes

All indexes in this schema are non-clustered, meaning each index exists as a separate structure from the heap table data. This was chosen because the schema serves many different query patterns with no single dominant access path that would justify restructuring a table into an Index-Organized Table. The heap-plus-multiple-indexes model allows the optimizer to freely choose whichever index is most selective for any given query.

LOCAL Partitioned Indexes

All indexes on partitioned tables are defined as `LOCAL`, meaning each index is divided into one segment per table partition which enables partition pruning. When a query filters on the partition key, Oracle scans only the relevant index segments rather than the entire index. The unpartitioned `author` dimension table uses standard global indexes, as no partition structure exists to align with.

Storage Structures:

The schema separates data and index storage into two dedicated tablespaces, each sized and configured for its specific workload.

Data Tablespace — `scikey_data`

The data tablespace is initialized at 10GB with autoextend enabled in 1GB increments up to a 25GB ceiling. The large initial allocation reflects the actual dataset size — 868k keywords, 750k+ documents with CLOB columns for abstracts and URLs, and several million rows across the bridge tables. The 1GB autoextend increment avoids frequent extension events, which can cause momentary I/O spikes, while the 25GB hard cap prevents runaway growth from consuming all available disk.

Index Tablespace— `scikey_index`

The index tablespace is physically separated from the data tablespace on a dedicated datafile. This is a standard Oracle best practice — separating indexes from table data means that a full table scan and a concurrent index scan do not compete for I/O on the same datafile. The tablespace starts at 2GB and extends to a 5GB maximum, appropriate given that indexes are a fraction of the size of the data they reference. All B-Tree and bitmap indexes defined in the schema explicitly target `scikey_index` via the `TABLESPACE scikey_index` clause.

Data Loading & Stress Testing

Data Gathering

HAL is an open archive where various types of publications can be stored along with associated metadata such as author name, classification, primary url, etc. Data is able to be retrieved via the public web API and stored locally.

Due to the unclarity of HAL's API documentation certain important details such as column type aren't listed for the column names provided. This is especially important when working with strings, as it's unclear the lengths of columns needed to store this input, unless tracked upon data collection. Additionally it's not always clear what a column represents as the explicit format is not given and descriptions are sparse if provided at all.

Using the previously described approach, data is gathered via a python script and formatted into a JSON File for bulk loading.

Bulk Loading

A connection is made to our oracle database from the local VM. This is accomplished via a python script that takes in and manipulates HAL research records. These records are pulled from a previously created large JSON file. The insertion into the Oracle database is done in a memory-efficient manner. Rather than loading the entire file into memory at once, it streams the JSON array in configurable chunks of 500 records, keeping memory consumption flat regardless of file size.

For each record, the script resolves and inserts data across all tables in the correct dependency order with document first, then keywords, authors, and organisms. Commits are issued per chunk rather than per record, balancing transaction overhead against the risk of data loss on failure. Any individual record that fails is rolled back in isolation and logged, allowing the rest of the chunk to proceed without interruption. For loading 750k documents, the script takes approximately 1 hour 15 minutes to complete.

Workload Simulation:

To evaluate the performance of the SciKey data warehouse under realistic analytical conditions, a workload simulation was implemented using a custom SQL*Plus script executed through a bash driver. The workload was designed to simulate a read-intensive environment, reflecting

common operations such as document retrieval, keyword searches, author lookups, multi-table joins, and aggregation queries.

The simulation was executed using a bash script that launches multiple SQL*Plus sessions concurrently. Each session connects to the database and executes the same workload independently, allowing the system to be tested under parallel user activity. This approach enables evaluation of how the database performs under concurrent access and helps identify potential bottlenecks in CPU and I/O usage.

```bash
#!/usr/bin/env bash
DB_USER="scikey"
DB_PASS="scikeycosc416"
DB_CONN="localhost:1521/pdb.orcl.ca:pooled"
SESSIONS=${1:-2500}
CONSOLIDATED_LOG="./logs/read_stress_consolidated.log"

echo "Starting read stress test: ${SESSIONS} parallel sessions"
echo "Start time: $(date '+%Y-%m-%d %H:%M:%S')"
echo "---"
START_TIME=$(date +%s)

for ((i=1; i<=SESSIONS; i++)); do
  sqlplus -s "${DB_USER}/${DB_PASS}@${DB_CONN}" @read_stress_test.sql > "./logs/read_stress_session_${i}_.log" 2>&1 &
done

wait

END_TIME=$(date +%s)
ELAPSED=$((END_TIME - START_TIME))

# Consolidate all session logs into one file
{
  echo "============================================"
  echo "READ STRESS TEST REPORT"
  echo "Start time : $(date -d @${START_TIME} '+%Y-%m-%d %H:%M:%S')"
  echo "End time   : $(date -d @${END_TIME}   '+%Y-%m-%d %H:%M:%S')"
  echo "Sessions   : ${SESSIONS}"
  echo "Elapsed    : ${ELAPSED}s"
  echo "============================================"
  echo ""
  for ((i=1; i<=SESSIONS; i++)); do

    if grep -q "ERROR" "./logs/read_stress_session_${i}_.log"; then
      echo "SESSION ${i}"
      cat "./logs/read_stress_session_${i}_.log"
      echo ""
    fi
  done
  echo "============================================"
  echo "END OF REPORT"
  echo "============================================"
} > "${CONSOLIDATED_LOG}"

rm ./logs/read_stress_session*

echo "---"
echo "Read stress test complete."
echo "End time: $(date '+%Y-%m-%d %H:%M:%S')"
echo "Sessions: ${SESSIONS}"
echo "Wall clock time: ${ELAPSED}s"
echo ""
echo "Per-session logs : read_stress_session_*.log"
echo "Consolidated log : ${CONSOLIDATED_LOG}"
```

Figure 2.

The SQL workload consists of twelve query patterns designed to exercise different access paths within the database. These include full table scans on the document table, indexed prefix searches on keyword.keyword_text and author.last_name, multi-table joins across document, doc_author, doc_keyword, and keyword, aggregation queries such as document counts per author and distinct author counts per organism, as well as CLOB access and correlated subqueries. Randomized keyword and author prefixes are generated during each iteration to ensure varied query execution and to avoid repeated access to the same data ranges.

Each query is executed inside a PL/SQL loop using SYS_REFCURSOR, where the cursor is opened and closed immediately after execution. This design ensures that the workload focuses on query execution performance rather than result output, reducing overhead and providing a more accurate representation of backend processing.

```oracle
SET PAGESIZE 0
SET FEEDBACK OFF
SET VERIFY OFF
SET HEADING OFF
SET TIMING ON
SET SERVEROUTPUT ON
SET TRANSACTION READ ONLY
WHENEVER SQLERROR CONTINUE

DECLARE
  v_loops NUMBER := 50;
  v_rc    SYS_REFCURSOR;

  -- Random-pick arrays for LIKE filters
  TYPE t_str_tab IS TABLE OF VARCHAR2(60) INDEX BY PLS_INTEGER;
  v_kw_prefixes   t_str_tab;
  v_auth_prefixes  t_str_tab;
  v_kw_idx         PLS_INTEGER;
  v_auth_idx       PLS_INTEGER;
BEGIN
  -- Seed the random generator
  DBMS_RANDOM.SEED(TO_NUMBER(TO_CHAR(SYSTIMESTAMP, 'FF6')));

  -- Keyword LIKE prefixes (varied enough to hit different index ranges)
  v_kw_prefixes(1)  := 'stress%';
  v_kw_prefixes(2)  := 'algorithm%';
  v_kw_prefixes(3)  := 'model%';
  v_kw_prefixes(4)  := 'network%';
  v_kw_prefixes(5)  := 'optim%';
  v_kw_prefixes(6)  := 'learn%';
  v_kw_prefixes(7)  := 'data%';
  v_kw_prefixes(8)  := 'simul%';
  v_kw_prefixes(9)  := 'comput%';
  v_kw_prefixes(10) := 'analy%';
  v_kw_prefixes(11) := 'control%';
  v_kw_prefixes(12) := 'system%';

  -- Author last-name LIKE prefixes
  v_auth_prefixes(1)  := 'S%';
  v_auth_prefixes(2)  := 'M%';
  v_auth_prefixes(3)  := 'B%';
  v_auth_prefixes(4)  := 'L%';
  v_auth_prefixes(5)  := 'D%';
  v_auth_prefixes(6)  := 'C%';
  v_auth_prefixes(7)  := 'R%';
  v_auth_prefixes(8)  := 'G%';
  v_auth_prefixes(9)  := 'P%';
  v_auth_prefixes(10) := 'T%';

  FOR i IN 1..v_loops LOOP

    -- Pick random prefixes for this iteration
    v_kw_idx   := TRUNC(DBMS_RANDOM.VALUE(1, 13));  -- 1..12
    v_auth_idx := TRUNC(DBMS_RANDOM.VALUE(1, 11));   -- 1..10
```

Figure 3.

```oracle
  -- Pick random prefixes for this iteration
  v_kw_idx   := TRUNC(DBMS_RANDOM.VALUE(1, 13));  -- 1..12
  v_auth_idx := TRUNC(DBMS_RANDOM.VALUE(1, 11));   -- 1..10

  -- Q1: Fact table scan - random page of documents
  OPEN v_rc FOR
    SELECT d.title, d.document_type, d.discipline
    FROM document d;
  CLOSE v_rc;

  -- Q2: Index scan - random keyword prefix (idx_keyword_text)
  OPEN v_rc FOR
    SELECT k.keyword_key, k.keyword_text, k.wikidata_id
    FROM keyword k
    WHERE k.keyword_text LIKE v_kw_prefixes(v_kw_idx);
  CLOSE v_rc;

  -- Q3: Index scan - random author last-name prefix (idx_author_name)
  OPEN v_rc FOR
    SELECT a.author_key, a.last_name, a.first_name, a.author_id_hal
    FROM author a
    WHERE a.last_name LIKE v_auth_prefixes(v_auth_idx);
  CLOSE v_rc;

  -- Q4: Two-table join - documents with keywords (random keyword prefix)
  OPEN v_rc FOR
    SELECT d.title, k.keyword_text
    FROM doc_keyword dk
    JOIN keyword  k ON dk.keyword_key  = k.keyword_key
    JOIN document d ON dk.document_key = d.document_key
    WHERE k.keyword_text LIKE v_kw_prefixes(v_kw_idx);
  CLOSE v_rc;

  -- Q5: Two-table join - documents with authors (random author prefix)
  OPEN v_rc FOR
    SELECT d.title, a.last_name, a.first_name, da.quality
    FROM doc_author da
    JOIN author   a ON da.author_key   = a.author_key
    JOIN document d ON da.document_key = d.document_key
    WHERE a.last_name LIKE v_auth_prefixes(v_auth_idx);
  CLOSE v_rc;

  -- Q6: Three-table join - author affiliations per document (random author prefix)
  OPEN v_rc FOR
    SELECT a.last_name, o.struct_name, d.title
    FROM author_organism ao
    JOIN doc_author da ON ao.document_key = da.document_key
                      AND ao.author_key   = da.author_key
    JOIN organism o    ON ao.organism_key  = o.organism_key
    JOIN author   a    ON da.author_key    = a.author_key
    JOIN document d    ON da.document_key  = d.document_key
    WHERE a.last_name LIKE v_auth_prefixes(v_auth_idx);
  CLOSE v_rc;

  -- Q7: Full star join - document + authors + keywords (random keyword prefix)
  OPEN v_rc FOR
    SELECT d.title, a.last_name, k.keyword_text, d.doi_id
    FROM document d
    JOIN doc_author  da ON d.document_key = da.document_key
    JOIN author       a ON da.author_key  = a.author_key
    JOIN doc_keyword dk ON d.document_key = dk.document_key
    JOIN keyword      k ON dk.keyword_key = k.keyword_key
    WHERE k.keyword_text LIKE v_kw_prefixes(v_kw_idx);
  CLOSE v_rc;
```

Figure 4.

```oracle_plsql
    -- Q7: Full star join — document + authors + keywords (random keyword prefix)
    OPEN v_rc FOR
      SELECT d.title, a.last_name, k.keyword_text, d.doi_id
      FROM document d
      JOIN doc_author  da ON d.document_key = da.document_key
      JOIN author       a ON da.author_key  = a.author_key
      JOIN doc_keyword dk ON d.document_key = dk.document_key
      JOIN keyword      k ON dk.keyword_key = k.keyword_key
      WHERE k.keyword_text LIKE v_kw_prefixes(v_kw_idx);
    CLOSE v_rc;

    -- Q8: Aggregation — authors by document count (random author prefix)
    OPEN v_rc FOR
      SELECT a.last_name, a.first_name, a.author_id_hal, COUNT(da.document_key) AS doc_cnt
      FROM doc_author da
      JOIN author a ON da.author_key = a.author_key
      WHERE a.last_name LIKE v_auth_prefixes(v_auth_idx)
      GROUP BY a.author_key, a.last_name, a.first_name, a.author_id_hal
      ORDER BY doc_cnt DESC;
    CLOSE v_rc;

    -- Q9: Aggregation — organisms by distinct author count
    OPEN v_rc FOR
      SELECT o.struct_name, o.hal_structure_id, COUNT(DISTINCT ao.author_key) AS auth_cnt
      FROM author_organism ao
      JOIN organism o ON ao.organism_key = o.organism_key
      GROUP BY o.organism_key, o.struct_name, o.hal_structure_id
      ORDER BY auth_cnt DESC;
    CLOSE v_rc;

    -- Q10: CLOB access — read abstracts matching random keyword prefix
    OPEN v_rc FOR
      SELECT DBMS_LOB.SUBSTR(d.abstract, 200, 1)
      FROM document d
      JOIN doc_keyword dk ON d.document_key = dk.document_key
      JOIN keyword      k ON dk.keyword_key = k.keyword_key
      WHERE d.abstract IS NOT NULL
        AND k.keyword_text LIKE v_kw_prefixes(v_kw_idx);
    CLOSE v_rc;

    -- Q11: Filtered star join — alternate keyword prefix each iteration
    OPEN v_rc FOR
      SELECT d.title, d.hal_id_s, k.keyword_text
      FROM document d
      JOIN doc_keyword dk ON d.document_key = dk.document_key
      JOIN keyword      k ON dk.keyword_key = k.keyword_key
      WHERE k.keyword_text LIKE v_kw_prefixes(MOD(v_kw_idx, 12) + 1);
    CLOSE v_rc;

    -- Q12: Correlated subquery — multi-affiliation authors (random name prefix)
    OPEN v_rc FOR
      SELECT a.last_name, a.first_name, a.author_id_hal
      FROM author a
      WHERE a.last_name LIKE v_auth_prefixes(v_auth_idx)
        AND (
          SELECT COUNT(DISTINCT ao.organism_key)
          FROM author_organism ao
          WHERE ao.author_key = a.author_key
        ) > 1;
    CLOSE v_rc;

  END LOOP;

  DBMS_OUTPUT.PUT_LINE('Read stress test complete: ' || v_loops
    || ' iterations x 12 queries = ' || (v_loops * 12) || ' total queries');
END;
/

EXIT
```

Figure 5.

The workload is executed in read-only mode to isolate analytical performance without introducing write contention. Multiple sessions are run in parallel, and logs are generated for each session and consolidated into a single report. This setup provides both high-level execution metrics and detailed session-level diagnostics, allowing for effective evaluation of performance tuning strategies such as partitioning, indexing, and materialized views.

Security Roles

Security in the SciKey data warehouse was implemented through a combination of encryption, auditing, and controlled access to ensure data protection and system accountability. The primary focus was on securing sensitive data at rest, monitoring database activity, and enforcing role-based user privileges.

To protect sensitive data, Oracle Transparent Data Encryption (TDE) was implemented. A password-protected TDE wallet securely stores the master encryption key, and encryption is applied to data at rest without requiring application changes. This ensures that database files and backups remain protected against unauthorized access.

Oracle Unified Auditing was configured to centrally log database activity, including user logins, data access, and administrative operations. This provides visibility into system behavior and supports monitoring and accountability. Auditing policies can be configured to track specific operations such as data modifications or access to critical tables.

Access control was enforced through role-based privilege assignment. A dedicated schema user handles application-level operations, while administrative privileges are restricted to system-level tasks. This separation follows the principle of least privilege, ensuring users only have access required for their role.

While these security mechanisms provide strong data protection and accountability, they introduce measurable performance overhead. Transparent Data Encryption (TDE) increases CPU usage during read and write operations due to encryption and decryption, particularly when working with large datasets. Unified Auditing generates additional redo log activity for each tracked operation, increasing I/O load and contributing to more frequent log switching under heavy workloads. Access control introduces minor overhead during query parsing as privileges are validated.

From an operational perspective, testing strategies were implemented to maintain system stability during tuning and deployment. Unit testing was used to validate individual components such as queries and materialized views, while regression testing ensured that performance optimizations did not negatively impact existing functionality. These results highlight the importance of balancing security and performance through selective encryption and targeted auditing policies.

Performance Tuning Results

Partitions & Indexes

Here are the results after implementing the previously mentioned partitioning and Indexing strategies seen via the compare periods report for a single heavy stress testing query.

Name	First Period Metric Ratio	Second Period Metric Ratio	First Period Value	Second Period Value	First Period Rate Per Second	Second Period Rate Per Second
DB cpu (seconds)			0.00	0.00	0.00	0.00
DB time (seconds)			168.72	130.42	1.76	1.12
db block changes			19,276.00	20,658.00	200.79	178.09
execute count			126,894.00	130,176.00	1,321.81	1,122.21
parse time elapsed (seconds)			2.10	4.48	0.02	0.04
physical reads			98.00	162.00	1.02	1.40
physical writes			32.00	58.00	0.33	0.50

Figure 6.

The compare periods report for a heavy stress test query shows a clear overall improvement after implementing partitioning and indexing. Total DB time dropped from 168.72 to 130.42 seconds which is roughly a 23% reduction. This is driven by Oracle's ability to prune partitions and resolve rows through indexes rather than scanning entire tables.

The one metric that increased was parse time, rising from 2.10 to 4.48 seconds. This is expected with partitioning and multiple indexes in place, the optimizer must evaluate partition pruning eligibility and compare index access paths before execution begins. This is a fixed upfront cost that is significantly outweighed by the savings at execution time, and can be further mitigated by execution plan caching on repeated queries.

Connection Pool

To improve connection scalability under high-concurrency workloads, Oracle Database Resident Connection Pooling (DRCP) was evaluated as part of the performance tuning process. Connection pooling reduces the overhead associated with repeatedly establishing and terminating database sessions by allowing multiple client requests to reuse a smaller set of pre-established connections.

A stress test was conducted using 5,000 parallel connection attempts to evaluate system behavior under heavy load. The number of connection brokers was varied to analyze its impact on performance. Without connection pooling (0 brokers), the system achieved 1,402 successful connections with a total execution time of 187 seconds. When connection pooling was enabled, performance improved significantly. The best result was observed with 2 connection brokers, where successful connections increased to 2,988 and execution time decreased to 109 seconds. Increasing the number of brokers beyond this point resulted in diminishing returns, with reduced connection success rates and increased execution time, indicating resource contention.

Further analysis was performed by varying the maximum connection pool size while keeping the number of brokers fixed at 2. The default pool size of 40 connections produced the best overall performance, achieving 2,988 successful connections with a relatively low execution time of 109 seconds. Reducing the pool size to 20 resulted in a decrease in successful connections (2,516), indicating that the pool was too small to handle the workload efficiently. Conversely, increasing the pool size to 60 significantly degraded performance, with successful connections dropping to 1,334 and execution time increasing to 179 seconds. This suggests that excessively large pool sizes introduce additional overhead and resource contention.

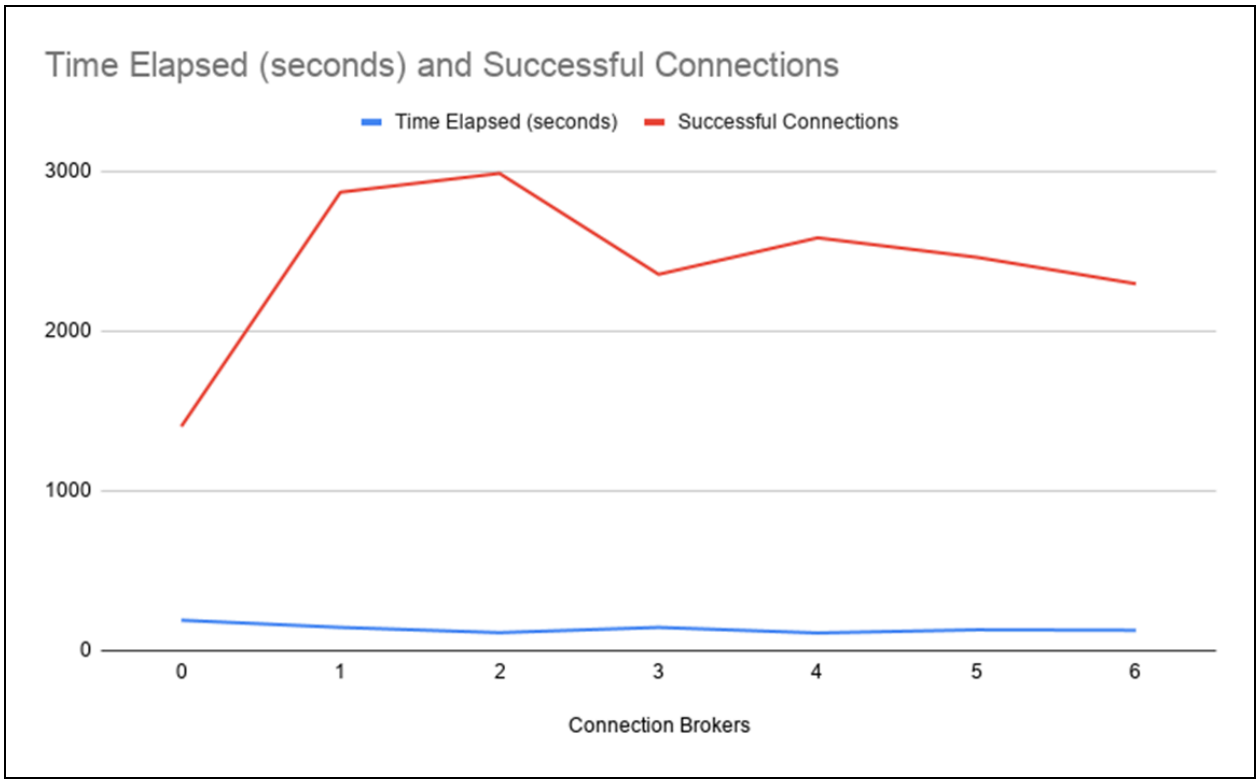


Figure 7.

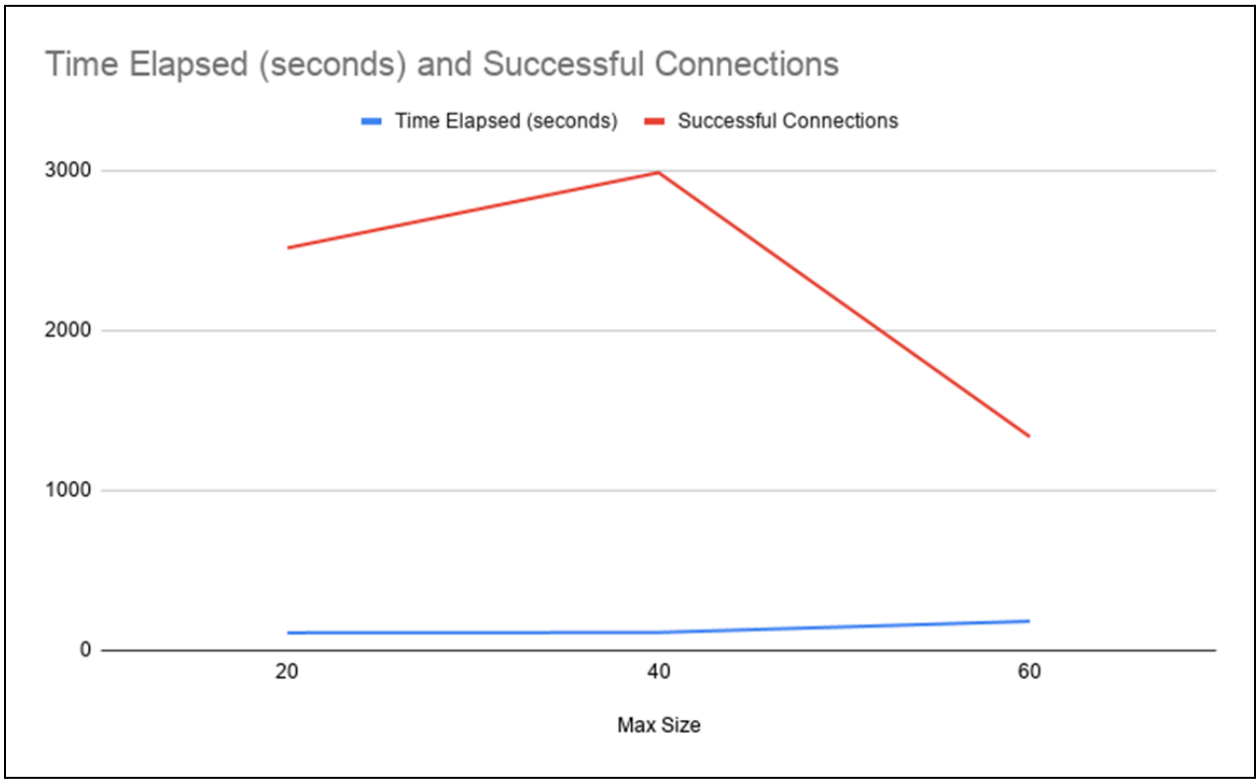


Figure 8.

Overall, the results demonstrate that connection pooling significantly improves system throughput and scalability under high-concurrency conditions. The optimal configuration for this workload was found to be approximately 2 connection brokers with the default pool size (40 connections). Deviation from this configuration, either by reducing or excessively increasing the pool size, resulted in reduced performance. This highlights the importance of tuning connection pool parameters to match workload characteristics.

Redo Log Writes

During heavy queries, a high volume of redo log switching was observed. The following query was run:

```
SELECT TO_CHAR(first_time, 'YYYY-MM-DD HH24:MI:SS') AS switch_time,

    ROUND((first_time - LAG(first_time) OVER (ORDER BY first_time)) * 24 * 60, 1) AS
mins_since_last

FROM   v$log_history
```

Revealing log churn, at as little as 90 seconds between switches. This high redo log turnover produces unnecessary checkpoints, reducing performance.

```
SWITCH_TIME         MINS_SINCE_LAST
------------------- ---------------
2026-03-28 19:09:35             1.6
2026-03-28 19:07:58             1.6
2026-03-28 19:06:22             1.9
2026-03-28 19:04:31             1.7
2026-03-28 19:02:49             1.8
2026-03-28 19:01:00             1.7
2026-03-28 18:59:21             1.7
2026-03-28 18:57:42             1.6
2026-03-28 18:56:09             1.5
2026-03-28 18:54:39

43 rows selected.

SQL>
```

Figure 9.

The redo log file size was queried and found to be only 300 MB. The logs were doubled to 600 MB to reduce the frequency of log switching.

```
SQL> select group#, bytes/1024/1024 as size_mb, status from v$log;

    GROUP#    SIZE_MB STATUS
---------- ---------- ----------------
         1        200 CURRENT
         2        200 INACTIVE
         3        200 INACTIVE

SQL>
```

Figure 10.

```
    GROUP#     SIZE_MB STATUS
---------- ---------- ----------------
         7        600 CURRENT
         8        600 UNUSED
         9        600 UNUSED

SQL>
```

Figure 11.

Following the resize, stress testing showed that the logs remain somewhat undersized under sustained heavy load, with switches still occurring every two to three minutes. However, during normal operational periods the current size is appropriate, with switches occurring several hours apart.

SWITCH_TIME	MINS_SINCE_LAST
2026-03-28 19:56:50	2.1
2026-03-28 19:59:05	2.3
2026-03-28 20:02:05	3
2026-03-28 20:04:23	2.3
2026-03-28 20:06:51	2.5
2026-03-28 20:09:36	2.8
2026-03-28 20:12:15	2.7
2026-03-28 20:14:51	2.6
2026-03-28 22:08:10	113.3
2026-03-29 06:00:37	472.5
2026-03-29 14:09:40	489.1

Figure 12.

Huge Pages

Research into optimizing memory for Oracle instances brought forth an interesting concept — increasing the page size on the operating system, allowing Oracle to bring in 2MB chunks from RAM. As discussed in Resource Allocation, this offers several performance benefits. An identical before-and-after stress test, with 1,000 concurrent sessions, 1,000 loops, and 10 queries per loop, resulted in an astonishing 84.8% reduction in DB CPU time.

Report Summary

Snapshot Set	DB Name	DB Id	Unique Name	DB Role	Edition	Release	Cluster	CDB	Host	Std Block Size
First (1st)	SCIKEY	2042188644	scikey	PRIMARY	EE	19.0.0.0.0	NO	YES	cosc416-ora-james.in.cosc-oc.ca	16384
Second (2nd)	SCIKEY	2042188644	scikey	PRIMARY	EE	19.0.0.0.0	NO	YES	cosc416-ora-james.in.cosc-oc.ca	16384

Snapshot Set	Instance	Inst num
First (1st)	scikey	1
Second (2nd)	scikey	1

Snapshot Set	Begin Snap Id	Begin Snap Time	End Snap Id	End Snap Time	Avg Active Users	Elapsed Time (min)	DB time (min)
1st	43	29-Mar-26 18:05:12 (Sun)	44	29-Mar-26 18:28:25 (Sun)	32.0	23.2	743.5
2nd	45	29-Mar-26 21:13:02 (Sun)	46	29-Mar-26 21:20:20 (Sun)	15.5	7.3	112.8
%Diff					*-51.7*	*-68.6*	*-84.8*

Figure 13.

Other performance increases of note are a massive reduction in parse time CPU seconds, at 86.86%, and a six-fold increase in physical reads per second.

Name	First Period Metric Ratio	Second Period Metric Ratio	First Period Value	Second Period Value	First Period Rate Per Second	Second Period Rate Per Second
parse time cpu (seconds)			706.02	92.78	0.51	0.21
parse time elapsed (seconds)			22,059.60	2,235.84	15.84	5.12
physical reads			1,434.00	2,756.00	1.03	6.31

Figure 14.

Another big win for Huge Pages was session logical reads, showing a 79.70% increase in rate per second.

Name	First Period Metric Ratio	Second Period Metric Ratio	First Period Value	Second Period Value	First Period Rate Per Second	Second Period Rate Per Second
session logical reads			215,810.00	333,660.00	154.92	763.52

Figure 15.